DOCKER

Name : Prathamesh Shingate

---

1. docker - -version

Command : **docker --version**
This command is used to see which Docker version is installed on your machine.

```
prathamesh@Prathamesh-Shingate:~/linux$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
prathamesh@Prathamesh-Shingate:~/linux$
```

2. docker info

command : **docker info**
This command gives detailed information about the Docker setup on your system and confirms that the Docker service is running.

```
prathamesh@Prathamesh-Shingate:~/linux$ docker info
Client:
 Version:    28.2.2
 Context:    default
 Debug Mode: false

Server:
 Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
 Images: 1
 Server Version: 28.2.2
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
 Logging Driver: json-file
 Cgroup Driver: systemd
 Cgroup Version: 2
 Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
 CDI spec directories:
  /etc/cdi
  /var/run/cdi
 Swarm: inactive
 Runtimes: io.containerd.runc.v2 runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version:
 runc version:
 init version:
 Security Options:
  apparmor
  seccomp
   Profile: builtin
  cgroupns
 Kernel Version: 6.14.0-37-generic
 Operating System: Ubuntu 24.04.3 LTS
 OSType: linux
 Architecture: x86_64
 CPUs: 12
 Total Memory: 15.34GiB
 Name: Prathamesh-Shingate
```

3. docker help

Command : **docker help**

This command shows all available Docker commands and helps you understand how to use Docker from the terminal.

```
prathamesh@Prathamesh-Shingate:~/linux$ docker help
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login       Authenticate to a registry
  logout      Log out from a registry
  search      Search Docker Hub for images
  version     Show the Docker version information
  info        Display system-wide information

Management Commands:
  builder     Manage builds
  container   Manage containers
  context     Manage contexts
  image       Manage images
  manifest    Manage Docker image manifests and manifest lists
  network     Manage networks
  plugin      Manage plugins
  system      Manage Docker
  trust       Manage trust on Docker images
  volume      Manage volumes

Swarm Commands:
  swarm       Manage Swarm

Commands:
  attach      Attach local standard input, output, and error streams to a running container
  commit      Create a new image from a container's changes
  cp          Copy files/folders between a container and the local filesystem
  create      Create a new container
  diff        Inspect changes to files or directories on a container's filesystem
  events      Get real time events from the server
  export      Export a container's filesystem as a tar archive
  history     Show the history of an image
  import      Import the contents from a tarball to create a filesystem image
  inspect     Return low-level information on Docker objects
  kill        Kill one or more running containers
  load        Load an image from a tar archive or STDIN
  logs        Fetch the logs of a container
  pause       Pause all processes within one or more containers
  port        List port mappings or a specific mapping for the container
  rename      Rename a container
```

4. docker pull

The docker pull command is used to download a **Docker image from a registry (like Docker Hub) to your local machine** so you can create and run containers from it.

Command : **docker pull <image-name>:version**

eg → docker pull nginx:latest

```
prathamesh@Prathamesh-Shingate:~/linux$ docker pull nginx:latest
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
prathamesh@Prathamesh-Shingate:~/linux$
```

5. docker images

This command lists all Docker images available on your system, showing their names, tags, and sizes. It helps you see what images you can use to create containers.

Command : **docker images**

```
prathamesh@Prathamesh-Shingate:~/linux$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    4af177a024eb   2 days ago     161MB
hello-world   latest    1b44b5a3e06a   5 months ago   10.1kB
prathamesh@Prathamesh-Shingate:~/linux$
```

6. docker run

The docker run command is used to create and start a container from a Docker image.

Command : **docker run -d - -name <custom-container-name> <image-name>:version**

- **-d** → Runs the container in detached mode, which means it runs in the background and doesn't block your terminal.
- **--name** → Assigns a custom name (newNginx) to your container instead of letting Docker give it a random name.

- **nginx:latest** → Uses the latest version of the Nginx image from Docker Hub to create the container.

```
prathamesh@Prathamesh-Shingate:~/linux$ docker run -d --name newNginx nginx:latest
6a18d9e8b94fb39d412a8101465365432acf8d9ca8def53bf627975d89f9030a
```

7. docker ps

This command lists all running containers on your system. It shows their IDs, names, status, and ports.

Command : **docker ps**

```
prathamesh@Prathamesh-Shingate:~/linux$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS     NAMES
fef9073f3425   nginx:latest   "/docker-entrypoint.…"   7 minutes ago   Up 7 minutes   80/tcp    newNginx
prathamesh@Prathamesh-Shingate:~/linux$ docker
```

8. docker ps -a

This command lists all containers, including running and stopped ones.

Command : **docker ps -a**

```
prathamesh@Prathamesh-Shingate:~/linux$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS                     PORTS     NAMES
6a18d9e8b94f   nginx:latest   "/docker-entrypoint.…"   14 minutes ago  Up 14 minutes              80/tcp    newNginx
c9eb13dd63e0   hello-world    "/hello"                 12 days ago     Exited (0) 12 days ago             nervous_jones
```

9. docker stop

The docker stop command is used to stop a running container.

Command : **docker stop <container-id/name>**

```
prathamesh@Prathamesh-Shingate:~/linux$ docker stop newNginx
newNginx
```

10. docker start

The docker start command is used to start a stopped container without creating a new one.

Command : **docker start <conntainer-id/name>**

```
prathamesh@Prathamesh-Shingate:~/linux$ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                     PORTS     NAMES
fef9073f3425   nginx:latest  "/docker-entrypoint.…"   17 minutes ago  Exited (0) 21 seconds ago            newNginx
c9eb13dd63e0   hello-world   "/hello"                 12 days ago     Exited (0) 12 days ago               nervous_jones
prathamesh@Prathamesh-Shingate:~/linux$ docker start newNginx
newNginx
prathamesh@Prathamesh-Shingate:~/linux$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS         PORTS     NAMES
fef9073f3425   nginx:latest  "/docker-entrypoint.…"   17 minutes ago  Up 4 seconds   80/tcp    newNginx
prathamesh@Prathamesh-Shingate:~/linux$
```

11. docker rm

The docker rm command is used to remove a stopped container from your system.

Command : **docker rm <container-id/name>**

eg→ docker rm newNginx

```
prathamesh@Prathamesh-Shingate:~/linux$ docker rm newNginx
newNginx
```

12. docker rmi

The docker rmi command is used to remove Docker images from your system.

Command : **docker rmi <image-name>**

eg → docker rmi nginx:latest

```
prathamesh@Prathamesh-Shingate:~/linux$ docker rmi nginx:latest
Untagged: nginx:latest
Untagged: nginx@sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Deleted: sha256:4af177a024eb8a1e43f4fb6c66735bb8260115cb5925a64f51673219bd97c144
Deleted: sha256:74821a6b71dfac14eaeb80c7c9e48130391306557f75228278054afe105cde75
Deleted: sha256:ae46c5037a8cc17f658f78b0e6523c9363ca800bf51f91f58cbc3aad8ffe53b5
Deleted: sha256:ec81248cdbc6b02bb24b1c32d4f59194a4a42aab3c81a42631efa1af7cba4f69
Deleted: sha256:774cc13f29089d7d0ebaaa291c5ac2d12841f2d70ab07a21c470d3cbdc11c059
Deleted: sha256:f1155703f3479bf6b295f5e17684156f82f815ca0798af8a2d08cf994411a055
Deleted: sha256:a455653294af11c547d0b3d2d769fd184f357cd5c0decbd1f89ffe9b380985aa
Deleted: sha256:e50a58335e1366e2581fe61794c1651afe2fe04e881e795aa166f24f4fc78d92
prathamesh@Prathamesh-Shingate:~/linux$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    1b44b5a3e06a   5 months ago   10.1kB
prathamesh@Prathamesh-Shingate:~/linux$
```

13. docker logs

The docker logs command is used to view the output (logs) of a container, whether it is running or stopped.

Command : **docker logs <container-name/id>**

eg → docker logs new-nginx

```
prathamesh@Prathamesh-Shingate:~/linux$ docker logs new-nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/01/15 13:18:06 [notice] 1#1: using the "epoll" event method
2026/01/15 13:18:06 [notice] 1#1: nginx/1.29.4
2026/01/15 13:18:06 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/01/15 13:18:06 [notice] 1#1: OS: Linux 6.14.0-37-generic
2026/01/15 13:18:06 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/01/15 13:18:06 [notice] 1#1: start worker processes
2026/01/15 13:18:06 [notice] 1#1: start worker process 29
2026/01/15 13:18:06 [notice] 1#1: start worker process 30
2026/01/15 13:18:06 [notice] 1#1: start worker process 31
2026/01/15 13:18:06 [notice] 1#1: start worker process 32
2026/01/15 13:18:06 [notice] 1#1: start worker process 33
2026/01/15 13:18:06 [notice] 1#1: start worker process 34
2026/01/15 13:18:06 [notice] 1#1: start worker process 35
2026/01/15 13:18:06 [notice] 1#1: start worker process 36
2026/01/15 13:18:06 [notice] 1#1: start worker process 37
2026/01/15 13:18:06 [notice] 1#1: start worker process 38
2026/01/15 13:18:06 [notice] 1#1: start worker process 39
2026/01/15 13:18:06 [notice] 1#1: start worker process 40
prathamesh@Prathamesh-Shingate:~/linux$
```

14. docker exec

The docker exec command is used to **run commands inside an already running container.** It’s like opening a terminal directly inside a running container so you can run commands .

Command : **docker exec -it <container-name> <any-shell>**

Shell : A shell is a program that lets you interact with the operating system by typing commands. Common examples are **bash, sh .**

Eg → docker exec -it new-nginx sh

```
prathamesh@Prathamesh-Shingate:~/linux$ docker exec -it new-nginx sh
# ls
bin  boot  dev  docker-entrypoint.d  docker-entrypoint.sh  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
#
```

To exit the interactive shell, you can use the exit command or press Ctrl+D.

15. docker stats

The docker stats command is used to display real-time resource usage statistics for running containers, including CPU, memory, network, and disk I/O.

Command : **docker stats**

eg → docker stats <container-name>

```
prathamesh@Prathamesh-Shingate: ~/linux
CONTAINER ID   NAME       CPU %    MEM USAGE / LIMIT    MEM %    NET I/O        BLOCK I/O    PIDS
79b56ea15071   new-nginx  0.00%    10.4MiB / 15.34GiB   0.07%    5.65kB / 126B  0B / 12.3kB  13
```

16. docker volume

A Docker volume is a storage space by Docker that allows containers to save and share data outside of the container's filesystem.

Data in a **volume is persistent**, meaning it remains safe even if the container is deleted or recreated.

Command : **docker run -it -v <host_path>:<container_path> --name <container_name> <image-name:tag> sh**

eg → docker run -it -v /home/prathamesh/mydata:/app --name vol-container ubuntu:latest sh

example :

Created a folder on host

```
prathamesh@Prathamesh-Shingate:~/linux$ mkdir -p /home/prathamesh/mydata
```

pulled a image from docker hub

```
prathamesh@Prathamesh-Shingate:~/linux$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
20043066d3d5: Pull complete
Digest: sha256:c35e29c9450151419d9448b0fd75374fec4fff364a27f176fb458d472dfc9e54
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Run a container and map the host folder to the container

-v → Mounts a host folder or Docker volume inside the container for persistent data.

-it → Runs the container interactively with a terminal so you can type commands inside it.

```
prathamesh@Prathamesh-Shingate:~/linux$ docker run -it -v /home/prathamesh/mydata:/app --name vol-container ubuntu:latest sh
# cd /app
# ls
# pwd
/app
# echo "This is the file from the vol-container for volume " > /app/test.txt
# ls
test.txt
# exit
```

And created a file called test.txt in inside the container /app folder. And exited the container .

file is saved inside the container, but physically it is on host machine in /home/prathamesh/mydata path.

```
prathamesh@Prathamesh-Shingate:~/linux$ cat /home/prathamesh/mydata/test.txt
This is the file from the vol-container for volume
```

Data written inside the container is visible on the host, and data added on the host is immediately available inside the container — **tested both ways (host ↔ container)**.

```
prathamesh@Prathamesh-Shingate:~/mydata$ touch newfile1.txt
prathamesh@Prathamesh-Shingate:~/mydata$ cd ..
prathamesh@Prathamesh-Shingate:~$ docker exec -it vol-container sh
Error response from daemon: container db49e5e96aaf02fb5144c1ba8d199c7270504b651cd31fc1af654c0857aff2d8 is not running
prathamesh@Prathamesh-Shingate:~$ docker start vol-container
vol-container
prathamesh@Prathamesh-Shingate:~$ docker exec -it vol-container sh
# pwd
/
# cd /app
# ls
newfile1.txt  test.txt
#
```

Even if the container is deleted, the data still remains on the host.

```
prathamesh@Prathamesh-Shingate:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS     NAMES
db49e5e96aaf   ubuntu:latest  "sh"                     41 minutes ago  Up 34 minutes            vol-container
79b56ea15071   nginx:latest   "/docker-entrypoint.…"   2 hours ago     Up 2 hours     80/tcp    new-nginx
prathamesh@Prathamesh-Shingate:~$ docker stop  vol-container
vol-container
prathamesh@Prathamesh-Shingate:~$ docker rm vol-container
vol-container
prathamesh@Prathamesh-Shingate:~$ cd /home/prathamesh/mydata
prathamesh@Prathamesh-Shingate:~/mydata$ ls
newfile1.txt  test.txt
prathamesh@Prathamesh-Shingate:~/mydata$
```

17. dockerfile :

Dockerfile contains set of instructions to build the docker image.

Docker Image is created from the Dockerfile.

Docker Container is a running instance of the image that serves the website.

Dockerfile Attributes Used :

FROM: Specifies the base image.

RUN : Executes commands while building the image.

COPY: Copies files from host to image.

EXPOSE: Declares the port used by the container.

CMD: Defines the command that runs when the container starts.

```
prathamesh@Prathamesh-Shingate:~/linux$ cat Dockerfile
FROM nginx:latest
RUN apt-get update
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Created the Docker image using the Dockerfile.

Eg →

Command **: docker build -t nginx-ig .**

-t : Assigns a name tag to the Docker image

. : Specifies the current directory

```
prathamesh@Prathamesh-Shingate:~/linux$ docker build -t nginx-ig .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  28.16kB
Step 1/5 : FROM nginx:latest
 ---> 4af177a024eb
Step 2/5 : RUN apt-get update
 ---> Using cache
 ---> 47e68d277090
Step 3/5 : COPY index.html /usr/share/nginx/html/index.html
 ---> a108703ce2cb
Step 4/5 : EXPOSE 80
 ---> Running in c9691e041cb4
 ---> Removed intermediate container c9691e041cb4
 ---> d1b16af460c2
Step 5/5 : CMD ["nginx", "-g", "daemon off;"]
 ---> Running in 288fb2eaac47
 ---> Removed intermediate container 288fb2eaac47
 ---> cb7f9dcd801d
Successfully built cb7f9dcd801d
Successfully tagged nginx-ig:latest
```

Now created the container from the docker image.

Command : **docker run -d -p 8080:80 --name nginx-container nginx-ig**

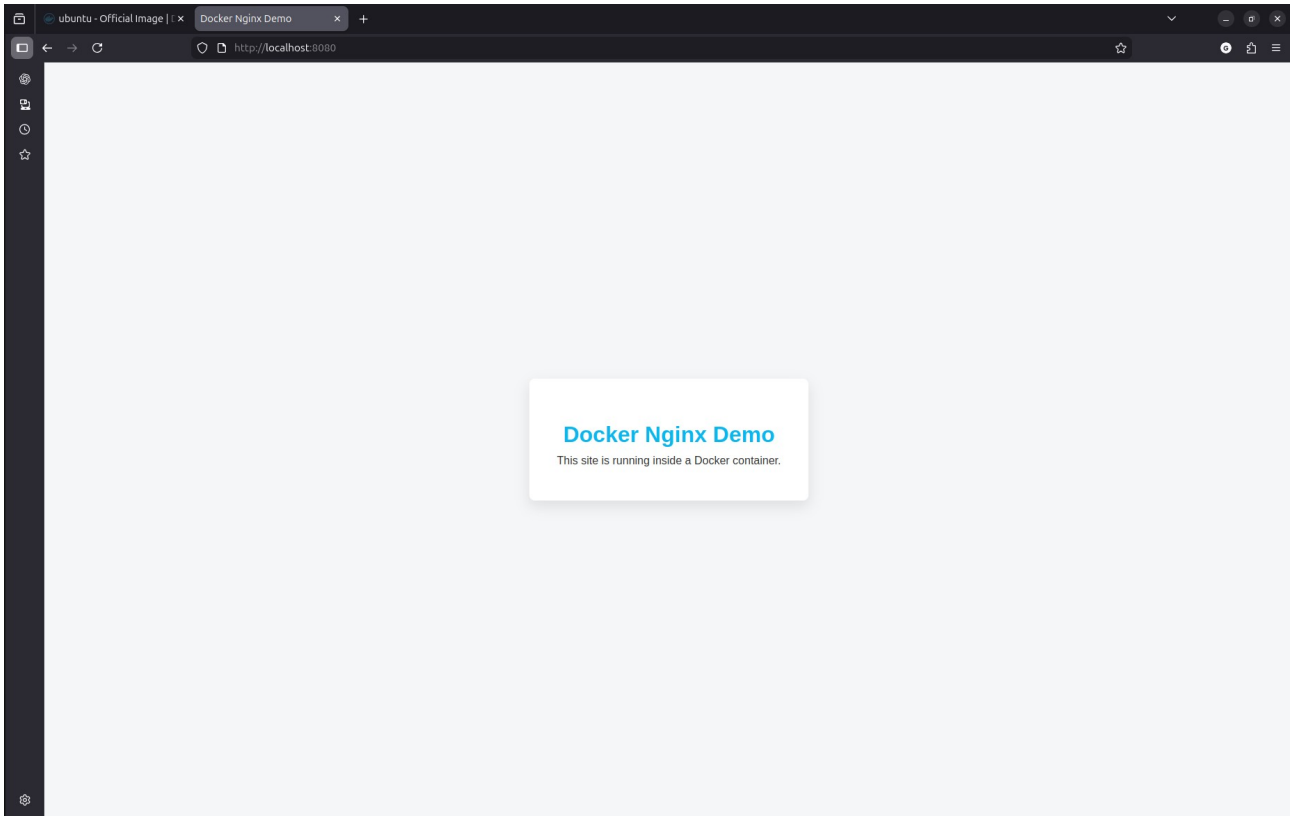
-d : Runs the container in detached (background) mode.

-p : Maps the hostport to container-port. Used for port mapping.

--name : Assigns a custom name to the container.

nginx-ig : Specifies the image used to create the container.

```
prathamesh@Prathamesh-Shingate:~/linux$ docker run -d -p 8080:80 --name nginx-container nginx-ig
be647bf966944715f95446f10a21783190f3d915019726dd7b594133318595f7
prathamesh@Prathamesh-Shingate:~/linux$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS                   PORTS                                      NAMES
be647bf96694   nginx-ig       "/docker-entrypoint.…"   36 seconds ago   Up 35 seconds            0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    nginx-container
79b56ea15071   nginx:latest   "/docker-entrypoint.…"   5 hours ago      Up 5 hours               80/tcp                                     new-nginx
c9eb13dd63e0   hello-world    "/hello"                 13 days ago      Exited (0) 4 hours ago                                              nervous_jones
```

Now our container is up and running , we can access website through →

http://localhost:8080